

Luồng cực đại và ghép đôi (Max Flow, Matching, Min Cut)

PHỤ THUỘC C/17 bắt buộc — toàn bộ chương này là mô hình đồ thị mở rộng; A/04 cho tính đúng của Ford–Fulkerson và cho chứng minh định lý luồng cực đại–lát cắt nhỏ nhất.

TÓM TẮT

Luồng cực đại là ví dụ mạnh nhất trong quyển sách này cho sức mạnh của việc mô hình hoá: rất nhiều bài không hề nói tới “dòng chảy” — phân công người vào việc, chọn dự án có lợi nhất, phân đoạn ảnh, kiểm tra một đội bóng còn cơ hội vô địch — đều quy về đúng một mô hình. Trung tâm lý thuyết của chương là định lý luồng cực đại bằng lát cắt nhỏ nhất, một sự trùng khớp giữa bài toán tối đa và bài toán tối thiểu; đó cũng là lý do luồng vừa cho lời giải vừa cho *chứng cứ* rằng lời giải là tối ưu. Chương tập trung vào mô hình hoá và các mẹo dựng đồ thị, còn phần cài đặt chỉ ở mức đủ dùng. Nếu chỉ nhớ một điều: khi đề bài có “sức chứa”, “mỗi thứ dùng một lần”, hoặc “chọn một trong hai phía sao cho ít mâu thuẫn nhất”, hãy nghĩ tới luồng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mạng luồng** — đồ thị có hướng với một đỉnh nguồn, một đỉnh đích, mỗi cạnh có sức chứa. Luồng là cách gán giá trị cho các cạnh sao cho không vượt sức chứa và mọi đỉnh trung gian có lượng vào bằng lượng ra.
- **đồ thị thặng dư** — đồ thị phụ ghi lại phần sức chứa còn lại và các cạnh ngược cho phép “rút lại” luồng đã đẩy — ý tưởng then chốt làm thuật toán đúng.
- **đường tăng luồng** — đường từ nguồn tới đích trong đồ thị thặng dư; còn đường như vậy thì luồng còn tăng được.
- **lát cắt** — cách chia tập đỉnh thành hai phần, một chứa nguồn một chứa đích; giá của lát cắt là tổng sức chứa các cạnh đi từ phần nguồn sang phần đích.
- **ghép đôi** — tập cạnh đôi một không chung đỉnh. Ghép đôi cực đại trên đồ thị hai phía quy về luồng cực đại với mọi sức chứa bằng 1.
- **phủ đỉnh** — tập đỉnh chạm tới mọi cạnh. Trên đồ thị hai phía, phủ đỉnh nhỏ nhất bằng ghép đôi lớn nhất — định lý König.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao luồng cực đại lại đúng bằng lát cắt nhỏ nhất — hai bài toán trông không liên quan gì tới nhau?
- Q2. Cạnh ngược trong đồ thị thặng dư cho phép “hoàn tác” luồng đã đẩy. Nếu bỏ chúng đi thì thuật toán hỏng ở đâu?
- Q3. Bài phân công n người vào n việc không nhắc gì tới dòng chảy. Vì sao nó lại là bài luồng?
- Q4. Ford–Fulkerson có thể không dừng với sức chứa vô tỉ. Điều đó nói gì về vai trò của “đại lượng giảm” trong chứng minh tính dừng?
- Q5. Vì sao trong phân đoạn ảnh, người ta lại đi tìm lát cắt nhỏ nhất chứ không tìm luồng?

Chương trước kết thúc bằng nhận xét rằng đường đi ngắn nhất hỏi về *một* đường. Nhưng rất nhiều bài toán thực tế hỏi về nhiều đối tượng cùng lúc, cạnh tranh nhau một nguồn lực có hạn: nhiều gói tin cùng chạy qua một mạng có băng thông giới hạn, nhiều người cùng được phân vào nhiều việc mà mỗi việc chỉ nhận một người, nhiều đường ống cùng dẫn nước qua các van có sức chứa khác nhau.

Mô hình luồng ra đời cho đúng lớp bài toán đó, và điều làm nó có giá trị vượt xa mục đích ban đầu là một hiện tượng lặp đi lặp lại: những bài toán trông chẳng liên quan gì tới dòng chảy hoá ra lại có cùng cấu trúc. Vì vậy chương này sẽ dành phần lớn trọng lượng cho câu hỏi *nhận diện và dựng mô hình*, và chỉ nói về thuật toán ở mức đủ để bạn chọn đúng và cài được.

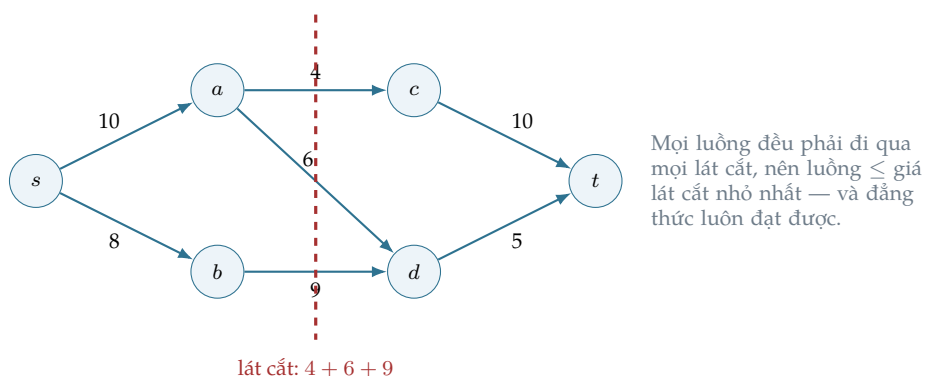
1.1 Luồng, lát cắt, và một sự trùng khớp đáng kinh ngạc

Bài toán: cho mạng với sức chứa trên mỗi cạnh, đẩy nhiều nhất bao nhiêu đơn vị từ nguồn tới đích?

Thuật toán cơ bản rất trực giác: tìm một đường từ nguồn tới đích còn chỗ trống, đẩy thêm luồng dọc đường đó, lặp lại. Điều duy nhất không hiển nhiên — và là ý tưởng then chốt của cả lý thuyết — là mỗi khi đẩy x đơn vị qua một cạnh, ta phải thêm một *cạnh ngược* sức chứa x vào đồ thị thặng dư. Cạnh ngược cho phép các bước sau “rút lại” quyết định cũ nếu nó hoá ra không tối ưu.

Vì sao cần? Vì nếu không có nó, thuật toán trở thành tham lam không hồi tiếc, và như chương C/15 đã chỉ ra, tham lam sai khi lựa chọn hôm nay phá huỷ khả năng tương lai. Đường đi đầu tiên có thể chiếm mất một cạnh hẹp mà lẽ ra nên để dành cho luồng khác. Cạnh ngược biến thuật toán tham lam thành thuật toán có khả năng tự sửa, và đó chính xác là điều làm nó tối ưu.

Kết quả trung tâm là *định lý luồng cực đại bằng lát cắt nhỏ nhất*: giá trị luồng lớn nhất đúng bằng giá của lát cắt nhỏ nhất. Chiều dễ hiểu ngay: mọi luồng đều phải đi qua mọi lát cắt, nên luồng không thể vượt giá của lát cắt nào. Chiều khó là chứng minh tồn tại một lát cắt *đạt* giá trị đó, và nó có một chứng minh rất đẹp: khi thuật toán dừng vì không còn đường tăng, hãy lấy tập đỉnh còn tới được từ nguồn trong đồ thị thặng dư — tập đó chính là một lát cắt có giá bằng luồng hiện tại.



Hình 1.1. Một mạng luồng và một lát cắt. Lát cắt vừa là chặn trên cho luồng vừa là *lời giải* của nhiều bài toán mô hình hoá (mục 4). Khi thuật toán dừng, tập đỉnh còn tới được từ s trong đồ thị thặng dư cho đúng một lát cắt đạt giá trị luồng.

TRỰC GIÁC

Vì sao một bài toán tối đa lại bằng một bài toán tối thiểu? Đây là hiện tượng *đối ngẫu*, và nó phổ biến hơn nhiều người nghĩ. Trực giác: luồng là “đẩy được bao nhiêu”, lát cắt là “chặn tồn ít nhất bao nhiêu”. Mỗi lát cắt là một *bằng chứng* chặn trên cho luồng; nếu tìm được lát

cắt có giá bằng đúng luồng hiện tại thì hai bên kẹp nhau và cả hai đều tối ưu. Đây là lý do luồng có tính chất hiếm: nó không chỉ cho lời giải mà còn cho *chứng cứ kiểm tra được* rằng lời giải là tối ưu. Cùng hiện tượng xuất hiện trong quy hoạch tuyến tính, trong lý thuyết trò chơi, và trong nhiều bài tối ưu tổ hợp khác.

1.2 Ba thuật toán và giả thiết của chúng

Ford–Fulkerson là khung chung: cứ tìm đường tăng bất kỳ. Chi phí phụ thuộc cách chọn đường: với sức chứa nguyên, mỗi vòng tăng ít nhất một đơn vị nên số vòng bị chặn bởi giá trị luồng — đó là *đại lượng giảm* theo nghĩa chương A/04. Nhưng nếu sức chứa là số vô tỉ thì các bước tăng có thể nhỏ dần vô hạn và thuật toán *không dừng*. Đây là một ví dụ hiếm và rất đáng nhớ về việc chứng minh tính dừng phụ thuộc vào giả thiết số học chứ không chỉ vào cấu trúc.

Edmonds–Karp chỉ định thêm một quy tắc: luôn chọn đường tăng *ngắn nhất* bằng duyệt rộng. Điều đó cho chặn $O(nm^2)$ độc lập với sức chứa, và giải quyết luôn vấn đề tính dừng. Bài học phương pháp rất đáng chú ý: một thuật toán không có đảm bảo tốt trở nên có đảm bảo chỉ nhờ *cổ định thứ tự lựa chọn*.

Dinic nhanh hơn nhiều và là lựa chọn thực tế: nó xây đồ thị phân tầng bằng duyệt rộng, rồi đẩy luồng chặn trên đồ thị đó bằng duyệt sâu. Chi phí $O(n^2m)$ nói chung, và $O(m\sqrt{n})$ trên mạng đơn vị của bài ghép đôi hai phía. Chỉ giả thiết mọi sức chứa bằng 1 chưa đủ để suy ra chặn này trên mạng tổng quát; cấu trúc vào/ra của các đỉnh trung gian cũng quan trọng[1]. Trong thực hành thi đấu, *Dinic* gần như luôn là lựa chọn đúng.

1.3 Ghép đôi hai phía và ba định lý anh em

Trường hợp riêng quan trọng nhất: đồ thị hai phía, mọi sức chứa bằng 1. Bài toán ghép đôi cực đại — phân công nhiều người vào nhiều việc sao cho số cặp lớn nhất — quy về luồng bằng cách thêm nguồn nối tới mọi đỉnh trái và đích nhận từ mọi đỉnh phải.

Ba định lý đi kèm, và chúng đáng thuộc vì chúng biến bài toán này thành bài toán khác:

Định lý König: trên đồ thị hai phía, ghép đôi lớn nhất bằng phủ đỉnh nhỏ nhất. Hệ quả: bài “chọn ít đỉnh nhất chạm mọi cạnh” — vốn NP-khó trên đồ thị tổng quát — lại giải được trong thời gian đa thức trên đồ thị hai phía.

Hệ quả về tập độc lập: tập độc lập lớn nhất bằng n trừ ghép đôi lớn nhất. Rất nhiều bài “chọn nhiều ô nhất trên bàn cờ sao cho không có hai ô xung khắc” rơi vào đúng đây.

Định lý Hall: tồn tại ghép đôi bão hoà phía trái khi và chỉ khi mọi tập con S của phía trái có số láng giềng ít nhất bằng $|S|$. Đây là công cụ để *chứng minh* một phân công là bất khả thi, chứ không chỉ để tìm phân công.

Với bài phân công có *chi phí* — mỗi người làm mỗi việc tốn một giá khác nhau, tìm phân công tổng chi phí nhỏ nhất — ta cần luồng chi phí nhỏ nhất hoặc thuật toán Hungarian[2]. Với $n \leq 20$, quy hoạch động mặt nạ bit ($C/16$) thường ngắn hơn và đủ nhanh; với n lớn hơn thì phải dùng thuật toán chuyên.

1.4 Mô hình hoá: nơi chương này thật sự trả công

Đây là mục quan trọng nhất. Dưới đây là các mẫu dựng mô hình hay gặp, kèm dấu hiệu nhận biết trên đề.

Mẫu 1 — phân công. Dấu hiệu: hai loại đối tượng, mỗi phần tử một loại ghép với một phần tử loại kia. Đồ thị hai phía, sức chứa 1.

Mẫu 2 — ràng buộc ở đỉnh. Dấu hiệu: “mỗi thành phố chỉ đi qua một lần”, “mỗi máy xử lý tối đa k việc”. Tách đỉnh thành hai nửa nối bởi một cạnh có sức chứa bằng đúng giới hạn đó — kỹ thuật đã nhắc ở C/17 và ở đây nó thành công cụ chính.

Mẫu 3 — chọn dự án có lợi nhất. Dấu hiệu: có các dự án cho lợi nhuận, mỗi dự án đòi hỏi một số thiết bị tốn chi phí, thiết bị dùng chung được. Đây là bài đóng lớn nhất, và nó quy về lát cắt nhỏ nhất: nối nguồn tới dự án với sức chứa bằng lợi nhuận, nối thiết bị tới đích với sức chứa bằng chi phí, nối dự án tới thiết bị nó cần với sức chứa vô hạn. Lợi nhuận lớn nhất bằng tổng lợi nhuận trừ lát cắt nhỏ nhất.

Mẫu 4 — chia hai phía sao cho ít mâu thuẫn nhất. Dấu hiệu: mỗi đối tượng phải được gán một trong hai nhãn; gán sai nhãn cho một đối tượng tốn một giá; hai đối tượng kề nhau mà khác nhãn cũng tốn một giá. Đây chính là mô hình lát cắt nhỏ nhất, và nó là nền của phân đoạn ảnh bằng cắt đồ thị.

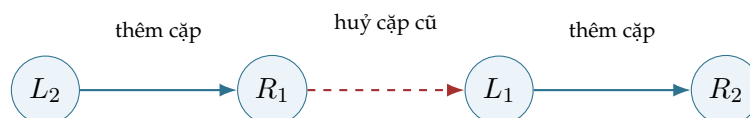
Mẫu 4 đáng được nói kỹ vì nó là cầu nối đẹp giữa chương này và thị giác máy tính. Khi ta muốn tách vật thể khỏi nền trong một bức ảnh, mỗi điểm ảnh phải nhận nhãn “vật” hoặc “nền”. Mỗi điểm ảnh có một chi phí gán nhãn dựa trên màu của nó, và hai điểm ảnh kề nhau có một chi phí nếu chúng nhận nhãn khác nhau — phần này thể hiện giả định rằng vùng ảnh thường liền mạch. Tổng chi phí đúng bằng giá của một lát cắt trong đồ thị mà đỉnh là điểm ảnh, nên tìm cách gán nhãn tốt nhất chính là tìm lát cắt nhỏ nhất. Đây là lý do câu hỏi mở đầu thứ năm có câu trả lời: người ta quan tâm tới *lát cắt* vì lát cắt chính là lời giải, còn luồng chỉ là phương tiện tính ra nó.

CẠM BẮY

Cạm bẫy phổ biến nhất khi dùng luồng không phải cài sai thuật toán mà là *dựng sai mô hình*, và cái sai đó im lặng: chương trình chạy, cho ra một con số, và con số đó trả lời một bài toán khác. Hai cách phòng. Thứ nhất, sau khi dựng xong, hãy kiểm tra bằng lời: “một đơn vị luồng trong mô hình này tương ứng với cái gì trong bài toán thật?” — nếu không trả lời được bằng một câu thì mô hình sai. Thứ hai, stress test với vét cạn trên dữ liệu rất nhỏ ($E/25$), vì mô hình sai gần như luôn lộ ra ở $n \leq 5$.

1.5 Cạnh ngược là quyền sửa một quyết định cũ

Cho hai người L_1, L_2 và hai việc R_1, R_2 ; các cặp hợp lệ là $L_1 R_1, L_1 R_2, L_2 R_1$. Nếu ghép $L_1 R_1$ trước, L_2 chưa có việc. Hình 1.2 cho thấy đường tăng dùng cạnh ngược để sửa cặp cũ.



Hình 1.2. Đường tăng xen kẽ $L_2 \rightarrow R_1 \rightarrow L_1 \rightarrow R_2$, lược bỏ nguồn và đích. Cạnh nét đứt là cạnh dư ngược của cặp đang ghép. Kết quả có hai cặp $L_2 R_1, L_1 R_2$, tăng từ 1 lên 2.

Khi đẩy Δ qua cạnh dư $u \rightarrow v$, giảm sức chứa dư cạnh đó và tăng cạnh đối ứng cùng lượng. Chỉ xoá bớt sức chứa chiều thuận sẽ biến thuật toán thành lựa chọn không thể sửa và có thể dừng ở luồng chưa cực đại. Trong cài đặt, lưu chỉ số cạnh đối ứng; hai cạnh gốc ngược chiều vẫn phải có các cặp cạnh dư riêng để xử lý chính xác đa cạnh. Sau khi chạy, kiểm sức chứa, bảo toàn luồng ở đỉnh trung gian và giá trị lát cắt từ các đỉnh còn tới được trong mạng dư.

1.6 Biên giới hiện nay: từ đường tăng tới tối ưu hoá liên tục

Dinic là lựa chọn thực hành, nhưng câu hỏi “luồng cực đại thật sự tốn bao nhiêu” có một lịch sử dài và vừa có bước ngoặt, và bước ngoặt ấy đáng kể lại vì nó đến từ một hướng hoàn toàn khác.

Suốt nửa thế kỷ, mọi cải tiến đều nằm trong khuôn tổ hợp: vẫn là đường tăng, luồng chặn, đẩy và gán nhãn, chỉ khác ở cách chọn và cách tổ chức dữ liệu. Đỉnh của dòng ấy là kết quả của Orlin[3], đưa luồng cực đại về $O(nm)$ cho mọi cặp n, m — con số mà nhiều người xem là điểm dừng tự nhiên của lối tiếp cận tổ hợp.

Bước ngoặt đến khi người ta ngừng coi luồng là bài toán tổ hợp. Nhìn từ tối ưu hoá, tìm luồng cực đại là giải một bài quy hoạch tuyến tính có cấu trúc rất đặc biệt, và người ta có thể đi tới lời giải bằng những bước *liên tục* — mỗi bước giải một bài toán dòng điện trên mạng, rồi sửa dần. Chen, Kyng, Liu, Peng, Probst Gutenberg và Sachdeva[4] hoàn thiện hướng này: luồng cực đại và luồng chi phí nhỏ nhất trên đồ thị có hướng với sức chứa và chi phí nguyên bị chặn đa thức, giải được trong thời gian $m^{1+o(1)}$ — *gần tuyến tính*, tức là chỉ hơn chi phí đọc dữ liệu vào một thừa số dưới mọi lũy thừa. Bài được trao giải bài báo xuất sắc tại FOCS 2022.

Hai điều đáng học ở đây, và không điều nào là “hãy cài thuật toán này”.

Thứ nhất, về phương pháp: bước nhảy có được nhờ *đổi ngôn ngữ* — rời khuôn tổ hợp sang khuôn tối ưu hoá liên tục — chứ không nhờ tinh chỉnh khuôn cũ. Và điều mĩ mai đẹp đẽ là cái làm cho hướng liên tục chạy được lại chính là cấu trúc dữ liệu động: mỗi bước liên tục chỉ rẽ nếu ta duy trì được thông tin về đồ thị đang thay đổi mà không tính lại từ đầu, đúng loại công việc của phần B. Lý thuyết thuật toán hiện đại có hình dạng như vậy: tối ưu hoá liên tục ở tầng trên, cấu trúc dữ liệu ở tầng dưới.

Thứ hai, về cách đọc kết quả: $m^{1+o(1)}$ giấu một thừa số dưới đa thức có thể rất lớn ở kích thước thật, nên trong mọi kỳ thi và gần như mọi hệ thống, Dinic vẫn là lựa chọn đúng, và ghép đôi hai phía vẫn nên dùng Hopcroft–Karp[5] với $O(m\sqrt{n})$. Nhưng kết quả này đổi một điều quan trọng cho *người mô hình hoá*: phát biểu lại bài toán của mình thành luồng chi phí nhỏ nhất giờ đây là một phép quy dẫn “gần như miễn phí” về mặt lý thuyết, nên rào cản còn lại chỉ là mô hình có đúng hay không — đúng thông điệp của mục 4.

1.7 Gốc rễ: mạng đường sắt Liên Xô và một bài toán về hôn nhân

Bài toán luồng cực đại có một nguồn gốc rất cụ thể và ít người biết. Năm 1955, trong bối cảnh Chiến tranh Lạnh, hai nhà nghiên cứu tại RAND Corporation là Harris và Ross viết một báo cáo phân tích năng lực vận chuyển của mạng đường sắt Liên Xô và các nước Đông Âu. Bài toán họ quan tâm có hai mặt: mạng này chuyển được tối đa bao nhiêu hàng, và — mặt được nhấn mạnh hơn — *cần cắt những tuyến nào, tốn ít nhất bao nhiêu, để chặn dòng vận chuyển*. Nghĩa là bài toán lát cắt nhỏ nhất được đặt ra như một bài toán quân sự, đồng thời với bài toán luồng cực đại.

Ford và Fulkerson, cũng tại RAND, đưa ra thuật toán và chứng minh định lý luồng cực đại bằng lát cắt nhỏ nhất vào năm 1956. Sự trùng khớp giữa hai bài toán — một của bên vận chuyển, một của bên phá hoại — vì thế không phải phát hiện lý thuyết thuần túy mà là câu trả lời cho một

câu hỏi thực tế được đặt ra từ cả hai phía cùng lúc. Đây là ví dụ mẫu mực cho việc bối cảnh sinh ra khái niệm: hai nửa của định lý đối ngẫu ứng với hai bên của một tình huống thật.

Nhánh ghép đôi thì cổ hơn và đến từ toán học thuần túy. König chứng minh định lý về ghép đôi và phủ đỉnh trên đồ thị hai phía năm 1931; Hall đưa ra điều kiện tồn tại ghép đôi bão hoà năm 1935, trong một bài toán thường được kể dưới cái tên “định lý hôn nhân”. Cả hai kết quả ra đời trước khi có máy tính và không nhằm mục đích thuật toán nào. Đến năm 1955, Kuhn kết hợp các ý tưởng của König và Egerváry thành một thuật toán cho bài toán phân công — và ông đặt tên nó là “phương pháp Hungary” để ghi công hai nhà toán học Hungary đó^[2], một cử chỉ hiếm thấy trong việc đặt tên thuật toán.

Cuối cùng, một liên hệ hiện đại đáng chú ý: đầu những năm 2000, giới thị giác máy tính phát hiện ra rằng bài toán gán nhãn ảnh với ràng buộc trơn có thể giải *chính xác* bằng lát cắt nhỏ nhất, thay vì bằng các phương pháp lặp không có đảm bảo. Điều này biến một thuật toán vận trù học của thập niên 1950 thành công cụ chủ lực cho phân đoạn ảnh và cho ước lượng độ sâu từ ảnh lập thể trong suốt một thập kỷ, cho tới khi học sâu thay đổi bức tranh.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1931 — König	Quan hệ giữa ghép đôi và phủ đỉnh trên đồ thị hai phía.	Định lý König; biến bài phủ đỉnh trên đồ thị hai phía thành bài đa thức.
1935 — Hall	Khi nào thì ghép được mọi phần tử của một phía?	Điều kiện Hall; công cụ để chứng minh <i>bất khả thi</i> , không chỉ để tìm lời giải.
1955 — Harris & Ross (RAND)	Năng lực mạng đường sắt Liên Xô, và cần cắt tuyến nào để chặn.	Bài toán luồng cực đại và lát cắt nhỏ nhất được đặt ra cùng lúc, từ hai phía của một tình huống thật.
1955 — Kuhn	Bài toán phân công người vào việc với chi phí.	Phương pháp Hungary, dựa trên kết quả của König và Egerváry.
1956 — Ford & Fulkerson	Cần thuật toán và cần biết khi nào dừng được.	Thuật toán đường tăng và định lý luồng cực đại bằng lát cắt nhỏ nhất — luồng vừa cho lời giải vừa cho chứng cứ tối ưu.
1970–72 — Dinic; Edmonds & Karp	Ford–Fulkerson có thể rất chậm hoặc không dừng.	Cổ định thứ tự chọn đường (ngắn nhất trước) cho chặn độc lập với sức chứa; Dinic thêm đồ thị phân tầng, cho $O(m\sqrt{n})$ trên mạng đơn vị.
Đầu thập niên 2000 — thị giác máy tính	Gán nhãn điểm ảnh với ràng buộc trơn, các phương pháp lặp không có đảm bảo.	Cắt đồ thị cho lời giải <i>chính xác</i> cho một lớp bài năng lượng; luồng trở thành công cụ chủ lực của phân đoạn ảnh và ảnh lập thể suốt một thập kỷ.

■ 1.8 Liên hệ ngang

■ Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Thị giác máy tính	Cắt đồ thị cho phân đoạn ảnh và ảnh lập thể	Mô hình mẫu 4 ở mục 4: chi phí gán nhãn cộng chi phí khác nhau giữa hai điểm kề. Điểm đáng nhớ: đây là một trong số ít bài năng lượng trong thị giác máy tính có lời giải <i>tối ưu toàn cục</i> trong thời gian đa thức.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Quy hoạch tuyến tính	Đối ngẫu: bài cực đại và bài cực tiểu có cùng giá trị tối ưu	Luồng-lát cắt là một trường hợp riêng đẹp của đối ngẫu tuyến tính. Ý tưởng chung rất mạnh: nghiệm của bài đối ngẫu là <i>chứng cứ</i> cho tính tối ưu của bài gốc.
Kinh tế học	Bài toán vận tải, phân bổ nguồn lực, thị trường ghép cặp	Cùng mô hình; ghép cặp ổn định (bệnh viện–bác sĩ nội trú, trường–học sinh) là họ hàng gần và đã được dùng trong các hệ thống phân bổ thật.
Mạng máy tính	Bảng thông, định tuyến đa đường, độ tin cậy mạng	Bài toán gốc. Số đường đi rời cạnh giữa hai đỉnh bằng luồng cực đại với mọi sức chứa bằng 1 — định lý Menger, một cách phát biểu khác của cùng kết quả.
Thể thao, lập lịch	Đội nào còn cơ hội vô địch	Bài toán loại trừ: dựng mạng phân phối các trận còn lại; nếu luồng không bão hoà thì đội đang xét chắc chắn bị loại. Ví dụ điển hình cho việc bài toán không có “dòng chảy” nào vẫn là bài luồng.
Sản xuất	Lập lịch máy, phân công ca	Ràng buộc ở đỉnh (mỗi máy tối đa k việc) xử lý bằng tách đỉnh; ràng buộc cận dưới (mỗi ca phải có ít nhất 2 người) xử lý bằng luồng có cận dưới.
SLAM và robot	Chọn tập ràng buộc nhất quán, phân công nhiệm vụ cho đội robot	Phân công nhiệm vụ nhiều robot là bài ghép đôi có trọng số — đúng bài toán Hungarian. Việc chọn tập quan sát nhất quán thì gần với các bài chọn tập con hơn ($C/15$), nhưng khi ràng buộc là “mỗi điểm mốc ghép với đúng một quan sát” thì lại quay về ghép đôi.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì mỗi lát cắt là một *chặn trên* cho mọi luồng — luồng nào cũng phải đi qua nó — nên luồng cực đại không vượt lát cắt nhỏ nhất. Chiều còn lại được chứng minh bằng cách xây dựng: khi thuật toán dừng vì không còn đường tăng, tập các đỉnh còn tới được từ nguồn trong đồ thị thặng dư tạo thành một lát cắt có giá đúng bằng luồng hiện tại. Hai bên kẹp nhau nên bằng nhau. Đây là hiện tượng đối ngẫu, và ý nghĩa thực dụng là luồng cho ta cả lời giải lẫn chứng cứ kiểm tra được rằng nó tối ưu.
- A2. Thuật toán trở thành tham lam không hối tiếc và sẽ cho kết quả dưới mức tối ưu. Đường tăng đầu tiên có thể chiếm một cạnh hẹp mà lẽ ra nên dành cho luồng khác, và không có cách nào sửa. Cạnh ngược cho phép các bước sau rút lại một phần quyết định cũ, biến thuật toán thành có khả năng tự sửa — đúng thứ mà chương C/15 nói là điều kiện để tham lam đúng, chỉ ở đây ta đạt được bằng cách thêm khả năng hối tiếc thay vì bằng chứng minh.
- A3. Vì bài phân công có đúng cấu trúc “mỗi bên dùng một lần”: dựng đồ thị hai phía, nguồn nối tới mọi người với sức chứa 1, mỗi người nối tới các việc họ làm được, mỗi việc nối tới đích với sức chứa 1. Sức chứa 1 trên các cạnh từ nguồn và tới đích chính là cách mã hoá ràng buộc “mỗi người một việc, mỗi việc một người”. Một đơn vị luồng tương ứng với một cặp được ghép — và câu này chính là bài kiểm tra ở khối *Cạm bẫy*.
- A4. Nó cho thấy tính dừng không phải là hệ quả tự nhiên của việc “mỗi bước đều cải thiện”. Với sức chứa nguyên, mỗi bước tăng ít nhất một đơn vị nên có một đại lượng nguyên không âm giảm — và đó là chứng minh dừng. Với sức chứa vô tỉ, các bước tăng có thể nhỏ dần theo cấp số nhân và tổng hội tụ về một giá trị nhỏ hơn tối ưu, nên thuật toán chạy mãi. Đại lượng giảm phải bị chặn dưới bởi một lượng dương cố định, không chỉ đơn thuần là “giảm”.
- A5. Vì trong bài phân đoạn ảnh, *lát cắt chính là lời giải*: nó nói mỗi điểm ảnh thuộc phía nào, tức là được gán nhãn gì. Luồng chỉ là phương tiện để tính ra lát cắt nhỏ nhất, và giá trị luồng chỉ là tổng chi phí. Đây là một chỗ đáng nhớ về mặt tư duy: khi mô hình hoá bằng luồng, hãy luôn hỏi bạn cần *giá trị* hay cần *cấu hình* — và nếu cần cấu hình thì nó thường nằm ở phía lát cắt.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy hình dung một hệ thống ống nước: một bể nguồn, một bể đích, và nhiều đoạn ống nối nhau, mỗi đoạn chịu được một lưu lượng tối đa. Câu hỏi là bơm được nhiều nhất bao nhiêu nước từ nguồn tới đích.

Cách làm rất tự nhiên: tìm một đường ống còn chỗ trống từ nguồn tới đích, bơm thêm nước vào đó, rồi lặp lại. Chỉ có một mẹo phải nhớ, và đó là điểm tinh tế duy nhất của cả chương: mỗi khi đã bơm nước qua một đoạn ống, hãy ghi nhớ rằng ta *có quyền rút lại* phần nước đó nếu về sau thấy nên dùng ống ấy cho luồng khác. Không có quyền rút lại thì lần bơm đầu tiên có thể lỡ chiếm mất một đoạn ống hẹp quan trọng, và ta bị kẹt ở kết quả không tốt nhất.

Điều đẹp nhất của bài toán này là một sự trùng khớp. Nếu bạn hỏi ngược lại — “muốn chặn hoàn toàn dòng nước thì phải cắt những đoạn ống nào cho ít tốn kém nhất” — thì câu trả lời của bài chặn đúng bằng câu trả lời của bài bơm. Trục giác thì rõ: mọi giọt nước đều phải đi qua chỗ hẹp nhất, nên chỗ hẹp nhất quyết định lượng nước. Điều đáng giá là chiều ngược lại cũng đúng, nên khi tìm ra cách chặn rẻ nhất là bạn có luôn bằng chứng rằng không thể bơm nhiều hơn.

Và đây là lý do mô hình này quan trọng hơn hẳn phạm vi ống nước. Bài phân công nhân viên vào công việc chính là bài này với mỗi ống chịu được đúng một đơn vị. Bài tách vật thể khỏi nền trong ảnh cũng là bài này: mỗi điểm ảnh phải chọn một trong hai phía, và “chi phí cắt” chính là chi phí đặt ranh giới. Khi bạn thấy đề bài nói tới sức chứa, hoặc “mỗi thứ dùng một lần”, hoặc “chia hai phía sao cho ít mâu thuẫn nhất” — hãy nghĩ tới ống nước.

Một câu: *Cắt chỗ hẹp nhất và bơm nhiều nhất là hai câu hỏi cho cùng một đáp số.*

Chỗ không rút gọn được: Việc dựng đúng mô hình cho một bài mới vẫn đòi kinh nghiệm; mẫu ở mục 4 chỉ phủ những dạng hay gặp, không phủ hết.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ý tưởng then chốt của thuật toán	Cạnh ngược trong đồ thị thặng dư: cho phép rút lại luồng đã đẩy, biến tham lam thành thuật toán tự sửa được.
••• Định lý luồng-lát cắt	Luồng cực đại bằng lát cắt nhỏ nhất; lát cắt là chặn trên, và tập đỉnh tới được từ nguồn khi thuật toán dừng cho lát cắt đạt đúng giá trị đó.
••• Dấu hiệu bài luồng trên đề	Có “sức chứa”; “mỗi thứ dùng đúng một lần”; “phân công hai loại đối tượng”; “chia hai phía sao cho ít mâu thuẫn nhất”.
••• Ghép đôi hai phía	Quy về luồng với mọi sức chứa 1; Dinic cho $O(m\sqrt{n})$. Một đơn vị luồng = một cặp được ghép.
••• Định lý König và hệ quả	Trên đồ thị hai phía: ghép đôi lớn nhất = phủ đỉnh nhỏ nhất; tập độc lập lớn nhất = n trừ ghép đôi lớn nhất.
••• Bốn mẫu mô hình hoá	Phân công; tách đỉnh khi ràng buộc ở đỉnh; chọn dự án (bài đóng lớn nhất) qua lát cắt; gán nhãn hai phía với chi phí trơn.
•• Vì sao Ford–Fulkerson có thể không dừng	Với sức chứa vô tỉ, các bước tăng nhỏ dần vô hạn; đại lượng giảm phải bị chặn dưới bởi một lượng dương cố định.
•• Edmonds–Karp	Luôn chọn đường tăng ngắn nhất (duyệt rộng) $\Rightarrow O(nm^2)$, độc lập với sức chứa. Bài học: cố định thứ tự lựa chọn tạo ra đảm bảo.
•• Định lý Hall	Ghép bão hoà phía trái tồn tại khi và chỉ khi mọi tập con S có ít nhất $ S $ láng giềng — dùng để chứng minh <i>bất khả thi</i> .
•• Bài phân công có chi phí	Luồng chi phí nhỏ nhất hoặc Hungarian; với $n \leq 20$ thì quy hoạch động mặt nạ bit thường ngắn hơn ($C/16$).
•• Cạm bẫy lớn nhất	Dựng sai mô hình, và sai im lặng. Kiểm bằng câu “một đơn vị luồng tương ứng với cái gì” và bằng stress test ở $n \leq 5$.
• Định lý Menger	Số đường đi rời cạnh giữa hai đỉnh bằng số cạnh ít nhất phải bỏ để ngắt liên thông — một cách phát biểu khác của định lý luồng-lát cắt.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
Luồng cực đại tốn bao nhiêu	Dòng tổ hợp dừng ở $O(nm)$ (2013); hướng tối ưu hoá liên tục cộng cấu trúc dữ liệu động cho $m^{1+o(1)}$ (2022). Thực hành vẫn là Dinic, và ghép đôi hai phía vẫn là Hopcroft–Karp.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên cài thuật toán luồng nào cho thi đấu?

| Dinic là một lựa chọn khởi đầu thực dụng. Nó đủ nhanh cho gần như mọi bài có giới hạn hợp lý, đặc biệt mạnh trên mạng ghép đôi hai phía nơi nó cho $O(m\sqrt{n})$. Edmonds–Karp đáng biết về mặt lý thuyết nhưng chậm hơn nhiều trong thực tế. Ford–Fulkerson thuần tuý thì dừng dừng vì chi phí phụ thuộc sức chứa. Hãy có sẵn một bản Dinic đã stress test trong thư viện riêng (A/02).

2. Làm sao xử lý ràng buộc “mỗi đỉnh chỉ đi qua một lần”?

| Tách đỉnh: mỗi đỉnh v thành $v_{\text{vào}}$ và v_{ra} , nối bởi một cạnh sức chứa 1 (hoặc bằng giới hạn của đỉnh). Mọi cạnh vào v nay nối tới $v_{\text{vào}}$, mọi cạnh ra khỏi v xuất phát từ v_{ra} . Kỹ thuật này chuyển ràng buộc từ đỉnh sang cạnh — nơi mô hình luồng biểu diễn được — và nó là một trong hai ba mẹo cần thuộc của chương.

3. Bài của tôi cần mỗi cạnh có cả cận dưới lẫn cận trên. Làm sao?

| Đó là bài luồng có cận dưới, và có một phép biến đổi chuẩn: với mỗi cạnh có cận dưới l , trừ l khỏi sức chứa và bù lại bằng cách thêm luồng cưỡng bức vào hai đầu qua một nguồn và đích phụ. Sau biến đổi, việc tồn tại luồng hợp lệ tương đương với việc mạng phụ bão hoà. Kỹ thuật này không khó nhưng nhiều chi tiết, nên hãy cài một lần cho kỹ và cất vào thư viện.

4. Vì sao bài chọn dự án lại là lát cắt chứ không phải luồng?

| Vì cái ta cần là *tập dự án được chọn*, tức là một cách chia đỉnh thành hai phía — đúng định nghĩa của lát cắt. Cách dựng: nguồn nối tới mỗi dự án với sức chứa bằng lợi nhuận, mỗi thiết bị nối tới đích với sức chứa bằng chi phí, và dự án nối tới thiết bị nó cần với sức chứa vô hạn. Cạnh vô hạn bảo đảm không thể chọn dự án mà không trả tiền thiết bị. Lợi nhuận lớn nhất bằng tổng lợi nhuận trừ giá lát cắt nhỏ nhất.

5. Khi nào bài ghép đôi *không* quy về luồng được?

| Khi đồ thị không phải hai phía. Ghép đôi cực đại trên đồ thị tổng quát cần thuật toán Blossom của Edmonds, phức tạp hơn hẳn và hiếm khi xuất hiện trong thi đấu. Đây cũng là chỗ ranh giới lý thuyết thú vị: nhiều bài dễ trên đồ thị hai phía lại khó hơn hẳn hoặc NP-khó trên đồ thị tổng quát — phủ định là ví dụ rõ nhất (D/23).

6. Tôi dựng mô hình xong nhưng không chắc đúng. Kiểm thế nào?

| Ba bước, theo thứ tự rẻ dần. Một, trả lời bằng lời câu “một đơn vị luồng ứng với gì” và “một lát cắt ứng với gì” — nếu áp ứng thì mô hình sai. Hai, thử tay một ví dụ $n = 2$ hoặc $n = 3$ và so với đáp án tính tay. Ba, stress test với vết cận trên $n \leq 5$. Bước ba tốn mười phút và bắt được gần như mọi lỗi mô hình — rẻ hơn nhiều so với ngồi đọc lại code.

7. Luồng chi phí nhỏ nhất khác luồng cực đại thế nào về mặt tư duy?

| Luồng cực đại hỏi “đẩy được bao nhiêu”; luồng chi phí nhỏ nhất hỏi “đẩy đúng lượng này với tổng chi phí nhỏ nhất”. Thuật toán chuẩn thay việc tìm đường tăng bất kỳ bằng việc tìm đường tăng *rẻ nhất* — tức là chạy Bellman–Ford hoặc Dijkstra với thế năng trên đồ thị thặng dư. Điều cần chú ý: đồ thị thặng dư có cạnh ngược với chi phí âm, nên Dijkstra thuần tuý không dùng được và phải chuyển đổi trọng số (C/17).

I Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
• Ghép đôi cực đại trên đồ thị hai phía	CSES “School Dance”[6]	Dựng mạng và chạy Dinic; giải thích ý nghĩa của mỗi đơn vị luồng.
•• Luồng cực đại cơ bản	CSES “Download Speed”[6]	Chú ý cạnh song song và cạnh hai chiều.
•• Số đường đi rời cạnh giữa hai đỉnh	CSES “Distinct Routes”[6]	Sức chứa 1; truy vết các đường sau khi có luồng.
•• Phủ đỉnh nhỏ nhất và tập độc lập lớn nhất trên đồ thị hai phía	Bài kinh điển	Dùng định lý König; dựng phủ đỉnh từ ghép đôi bằng cách duyệt xen kẽ.
•• Đặt quân xe trên bàn cờ có ô cấm sao cho không ăn nhau	Bài kinh điển	Hàng và cột là hai phía — một bài mà mô hình hoá là toàn bộ lời giải.
••• Bài chọn dự án có lợi nhuận lớn nhất	Bài kinh điển	Lát cắt nhỏ nhất với cạnh vô hạn; tính lợi nhuận bằng tổng trừ lát cắt.
••• Phân công với chi phí, $n \leq 200$	Bài kinh điển	Luồng chi phí nhỏ nhất hoặc Hungarian; so với quy hoạch động mặt nạ bit ở $n \leq 20$.
••• Đội bóng nào còn cơ hội vô địch	Bài kinh điển	Bài loại trừ thể thao; ví dụ đẹp nhất cho việc bài không có “dòng chảy” nào vẫn là bài luồng.
••• Phân đoạn ảnh nhị phân bằng lát cắt nhỏ nhất	Bài tập ứng dụng	Dựng chi phí gán nhãn và chi phí trơn; chạy trên ảnh nhỏ và xem kết quả bằng mắt.
••• Luồng có cận dưới trên mỗi cạnh	Bài thi đấu kinh điển	Phép biến đổi chuẩn; cài một lần cho kỹ rồi cắt vào thư viện riêng.

Kết chương và đọc tiếp

Luồng là chương mà công sức bỏ ra chủ yếu nằm ở việc *dựng mô hình*, không ở việc cài thuật toán. Ba điều đáng mang theo: cạnh ngược là ý tưởng làm cho thuật toán tự sửa được; định lý luồng–lát cắt cho ta cả lời giải lẫn chứng cứ tối ưu; và bốn mẫu mô hình ở mục 4, đặc biệt mẫu “gán nhãn hai phía với chi phí trơn” vì nó xuất hiện ở những nơi bất ngờ nhất.

Hai chương còn lại của phần C là hai công cụ cắt ngang, dùng được cho mọi chương trước. Chương C/19 nói về một kỹ thuật có tỉ lệ “biến bài khó thành bài dễ” cao nhất trong cả quyển sách: khi bài toán tối ưu khó, hãy đổi nó thành bài toán kiểm tra và tìm nhị phân trên đáp án.

I Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Edmonds & Karp, “Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems”	Journal of the ACM, 1972	Đọc để thấy một thuật toán không có đảm bảo trở nên có đảm bảo chỉ nhờ cố định thứ tự chọn đường tăng.
nền tảng	Hopcroft & Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs”	SIAM Journal on Computing, 1973	Thuật toán ghép đôi nên dùng trong thực hành; đọc phần lập luận về các pha đường tăng ngắn nhất.
tra khi cần	Orlin, “Max Flows in $O(nm)$ Time, or Better”	STOC, 2013	Điểm dừng của dòng thuật toán tổ hợp; đọc phần mở đầu để có bản đồ lịch sử các cận.
tra khi cần	Chen, Kyng, Liu, Peng, Probst Gutenberg, Sachdeva, “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time”	FOCS, 2022 (Best Paper)	Chỉ cần đọc mục giới thiệu: nó nói rõ vì sao hướng liên tục thắng và cấu trúc dữ liệu động đóng vai trò gì.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
tra khi cần	Ahuja, Magnanti, Orlin, <i>Network Flows</i>	Prentice-Hall, 1993	Sách tra cứu cho các biến thể mô hình hoá: luồng chi phí nhỏ nhất, luồng có cận dưới, bài toán vận tải.

Nguồn

- [1] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [2] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [3] J. B. Orlin, “Max Flows in $O(nm)$ Time, or Better,” in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 765–774.
- [4] L. Chen, R. Kyng, Y. P. Liu, et al., “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time,” in *Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Best Paper Award, 2022.
- [5] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [6] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).